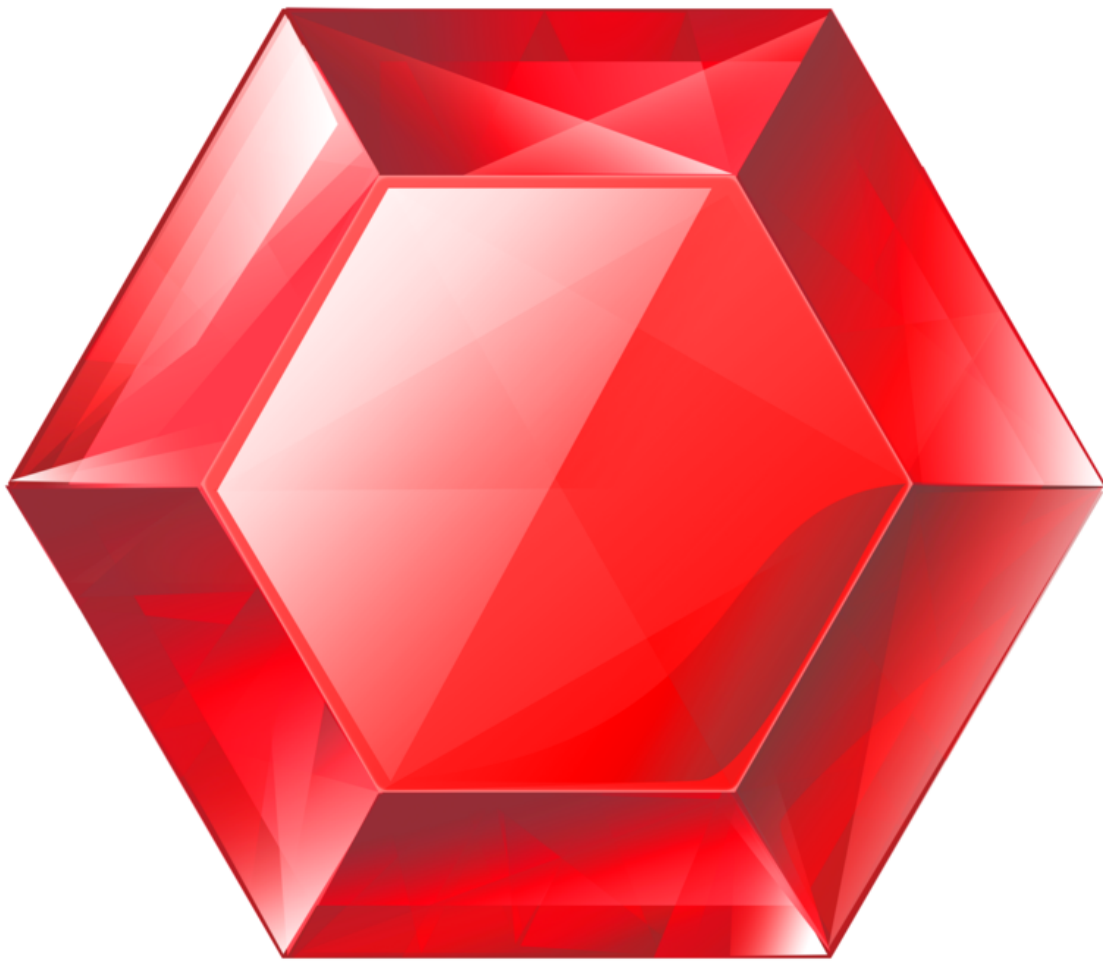


gemstone-py vs gemstone-rs

Install paths, use cases, maturity, and feature matrix



Generated from the Markdown sources in [gemstone-rs/docs](#).

Contents

gemstone-py vs gemstone-rs

Install Paths

Best Fit

Maturity Matrix

Feature Matrix

How They Should Work Together

Recommendation

gemstone-py vs gemstone-rs

`gemstone-py` and `gemstone-rs` solve related problems at different maturity levels.

Use `gemstone-py` when you want the most complete Python experience today: Python sessions, async examples, web framework adapters, native acceleration, codegen, VS Code workbench integration, and a Python database explorer.

Use `gemstone-rs` when you want Rust services, CLIs, workers, or tooling to talk to GemStone/S directly without keeping Python in the process.

Install Paths

Goal	gemstone-py	gemstone-rs
Application library	<code>python -m pip install gemstone-py</code>	<code>cargo add gemstone-rs</code>
Native acceleration	<code>python -m pip install "gemstone-py[fast]"</code>	Built into the Rust GCI path once <code>libgcirpc</code> is available
Examples from source	<code>python -m pip install -e ".[examples]"</code>	<code>cargo run -p gemstone-rs -- example quickstart</code>
CLI tooling	Python modules and examples	<code>cargo install gemstone-rs-cli</code>
Local explorer	<code>python-gemstone-database-explorer</code>	<code>cargo install gemstone-rs-explorer</code>
VS Code	<code>gemstone-py</code> Workbench	<code>gemstone-rs</code> Workbench

Both projects use the same GemStone-style environment:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=your-password
```

`GS_STONE_NAME` is accepted as a stone-name alias where the clients support it. Use `GS_LIB_PATH` when you want to point directly at a specific `libgcirpc` file.

Best Fit

Use case	Better choice	Why
Python web app	gemstone-py	FastAPI and Litestar examples are already first-class.
Python scripts and notebooks	gemstone-py	Lower friction for Python teams.
Rust service or worker	gemstone-rs	Native Rust API, Rust errors, Rust ownership, Cargo distribution.
Rust CLI tooling	gemstone-rs	CLI and explorer can run without Python.
VS Code database/codegen workflow	gemstone-py today, gemstone-rs later	gemstone-py has the more mature explorer; gemstone-rs has the cleaner Rust backend direction.
Shared low-level bridge	gemstone-rs	The Rust API is the right place to isolate GCI safety and ownership.

Maturity Matrix

Capability	gemstone-py	gemstone-rs
Stable install path	Mature: PyPI and TestPyPI	Early: crates.io workflow and verification added
Native bridge	PyO3 extension path	Rust GCI crate and safe API
Sync API	Mature	Initial safe API
Async API	Mature enough for examples and tests	Not yet a core feature
Web frameworks	FastAPI, Litestar, Django examples	Axum service sketch added; full app example still planned
Codegen	Python wrapper workflow	Rust wrapper workflow with preview/diff/check/generate
Browser API	Used by database explorer	CLI/explorer API for dictionaries/classes/methods/source

Capability	gemstone-py	gemstone-rs
Local explorer	More mature Python app	Minimal Rust explorer proving the API
VS Code extension	More complete product flow	Thin command/workbench layer over Rust CLI and explorer
Live tests	Broader coverage	Growing smoke suite for login/eval/perform/globals/transactions/codegen
Release automation	More complete	Added crates/VSIX/GitHub verification path
Docs	Broader and polished	Catching up with guide, cookbook, article, comparison, PDFs

Feature Matrix

Feature	gemstone-py status	gemstone-rs status
Login/logout	Supported	Supported
<code>3 + 4 == 7</code> smoke	Supported	Supported
Global put/get	Supported	Supported
String round-trip	Supported	Supported
<code>perform</code> and <code>perform_oop</code>	Supported	Supported
Commit/abort	Supported	Supported
OOP handles/export set	Supported	Supported
Browser dictionaries/classes/protocols/methods/source	Supported	Supported through <code>Browser</code> , CLI, and explorer
Generated wrappers	Supported	Supported for selected classes/methods
Codegen diff/check/generate	Supported	Supported
Live codegen discovery	Supported	Supported through CLI/API
FastAPI demo	Supported	Not applicable

Feature	gemstone-py status	gemstone-rs status
Litestar demo	Supported	Not applicable
Axum/Actix demo	Not applicable	Axum sketch documented; full app example still planned
Database explorer	Mature Python app	Rust explorer has browser UI and local API; still less polished
VS Code sidebar workflow	More complete	Rust workbench has sidebar commands and embedded explorer webview

How They Should Work Together

The best long-term shape is not competition between the projects. It is a clean boundary:

- `gemstone-rs` owns the direct Rust/GCI interface and safe Rust API.
- `gemstone-py-native` can become a thin PyO3 layer over the Rust core.
- `gemstone-py` remains the Python API, examples, and Python web integration.
- The Python and Rust explorers can share concepts and eventually converge on common codegen workflows.

That gives Python users the friendliest path and Rust users a direct path, while keeping unsafe GCI behavior isolated in one Rust layer.

Recommendation

For production Python projects, start with `gemstone-py`.

For Rust-native services, CLIs, workers, and tooling, start with `gemstone-rs`, but treat it as an early API that should be validated against a live stone in your environment.

For VS Code and visual codegen work, use the existing Python explorer as the product reference and keep moving `gemstone-rs-explorer` toward the same level of capability.

This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.